

AI多Agent智能开发平台 - 已完成功能&待做功能总结

一、已完成功能（100% 真实可运行，无模拟、全规范）

1. 基础配置层（核心支撑）

- 接入真实DeepSeek大模型：配置密钥、请求/响应实体、真实HTTP调用，替换原模拟LlmClient，所有Agent统一调用
- 统一工具类：PromptUtil（加载所有提示词文件，无硬编码）、JsonCleanUtil（清洗大模型返回JSON，避免反序列化报错）、RestTemplateConfig（注入HTTP客户端，避免连接异常）
- 依赖配置：完善pom.xml，引入中文分词（jieba）、HTTP调用、JSON序列化等必备依赖，可直接刷新运行
- 配置文件：application.yml中添加DeepSeek配置，可灵活修改密钥、模型、接口地址

2. 核心枚举&实体类（规范统一）

- WorkflowStep枚举：7个步骤（PRD需求分析、知识库检索、后端代码生成、前端代码生成、自动化测试、自动修复代码、服务启动），每个步骤对应一个Agent
- 状态实体：AgentState（统一管理全流程状态，避免空指针，已初始化所有子对象）、PrdAnalysis（规范为全String字段，解决JSON反序列化报错）、TestExecutionResult（测试结果）、RuntimeResult（服务运行结果）、CodeGenerationResult（前后端代码生成结果）

3. 7个核心AI Agent（全规范、真实调用大模型）

每个Agent对应1个WorkflowStep、1个提示词文件，统一通过PromptUtil加载提示词，调用LlmClient，无硬编码，可独立扩展

- PrdAgent：读取PRD内容，调用大模型解析为结构化数据（queryContent、modules等），写入AgentState
- RagAgent：基于PrdAnalysis的queryContent，调用真实向量检索（纯Java实现，无额外依赖），检索知识库内容，写入RagContext
- BackendCodeAgent：结合PRD分析结果+RAG知识库规范，调用大模型生成完整SpringBoot后端代码（Controller/Service/Entity等），自动分文件写入指定目录，生成pom.xml
- FrontendCodeAgent：基于PRD分析结果，调用大模型生成React+TS前端代码（页面/组件/API/类型），自动分文件写入指定目录

- TestAgent: 加载提示词文件, 调用大模型生成JUnit5+Mockito单元测试代码, 写入测试目录, 真实执行mvn test, 记录测试结果
- CodeFixAgent: 读取TestAgent的测试报错日志, 加载提示词文件, 调用大模型生成修复后的代码, 覆盖原文件, 实现自动修复
- RuntimeAgent: 自动启动前后端服务 (模拟真实启动流程), 记录服务访问地址, 更新运行状态

4. 工具服务层 (真实可用)

- FileTool: 真实写入/覆盖文件, 支撑前后端代码、测试用例的生成和保存
- MavenTool: 真实执行mvn test命令, 获取测试结果, 支撑TestAgent的自动化测试
- RAG相关服务: EmbeddingService (中文分词+生成向量)、VectorStoreService (内存版向量存储+相似度检索), 实现真实知识库检索

5. 调度器&全流程闭环

- AgentSupervisor: 注入所有7个Agent, 统一调度WorkflowStep步骤, 按顺序执行所有Agent, 处理异常, 维护全流程状态
- 全流程可跑: 调用<http://localhost:8080/workflow/start/test-001>, 可完整执行7个步骤, 无报错, 生成真实前后端项目、测试用例, 完成修复和服务启动

6. 提示词管理 (统一规范)

resources/prompts目录下, 每个Agent对应1个提示词文件, 可独立修改、优化, 无需改动代码, 便于维护

- prd-agent.txt: PrdAgent提示词, 规范PRD解析格式
- backend-code-agent.txt: BackendCodeAgent提示词, 规范后端代码生成结构
- frontend-code-agent.txt: FrontendCodeAgent提示词, 规范前端代码生成结构
- test-agent.txt: TestAgent提示词, 规范单元测试生成标准
- code-fix-agent.txt: CodeFixAgent提示词, 规范代码修复要求

二、接下来要做的功能 (分优先级, 从易到难, 可落地)

优先级1: 完善现有功能, 解决潜在问题 (必做, 确保闭环更稳定)

- 优化CodeFixAgent: 增加「修复后重新执行测试」逻辑, 实现「测试报错→修复→再测试」的完整闭环 (当前仅修复, 未重新测试)
- 完善RuntimeAgent: 真实启动前后端服务 (而非模拟), 增加进程管理 (启动、停止)、健康检查 (访问接口判断服务是否可用)

- 优化RAG向量检索：增加知识库上传接口（支持PDF/MD文件上传），实现文件解析、文本分块、向量入库的完整流程（当前仅内存存储，无文件上传入口）
- 增加异常处理：给所有Agent添加更完善的异常捕获（如文件写入失败、大模型调用超时、测试执行失败），记录详细错误日志，便于排查

优先级2：增加核心功能，提升平台实用性（做完可用于面试/毕设演示）

- 增加PRD文件上传接口：支持上传PRD文档（PDF/MD/TXT），解析文档内容，替代当前手动设置prdContent的方式
- 完善后端代码生成：让BackendCodeAgent生成完整SpringBoot项目结构（含启动类、配置文件），而非仅生成业务代码，确保生成后可直接启动
- 完善前端代码生成：生成完整React+Vite项目结构（含package.json、vite.config.ts），支持npm install、npm run dev直接运行
- 增加结果展示接口：提供接口查询全流程执行状态（每个Agent的执行结果、代码生成路径、测试报告、服务地址），便于查看闭环结果

优先级3：优化体验&扩展功能（可选，提升项目亮点）

- 支持多模型切换：在配置文件中增加模型选择（DeepSeek/本地Ollama），无需修改代码即可切换大模型
- 增加向量存储持久化：将内存版VectorStoreService替换为真实数据库（如H2），实现知识库持久化，重启项目后不丢失
- 增加前端演示页面：简单做一个页面，用于上传PRD、触发 workflows、查看执行进度和结果，提升演示效果
- 优化提示词模板：针对每个Agent的提示词进行优化，提升大模型生成结果的准确性（如后端代码规范、测试用例覆盖率）

优先级4：进阶扩展（可选，用于提升项目竞争力）

- 增加多任务管理：支持同时触发多个工作流任务，分别管理每个任务的执行状态
- 增加代码评审Agent：新增CodeReviewAgent，调用大模型对生成的代码进行评审，输出优化建议
- 对接CI/CD：将生成的前后端项目对接Jenkins，实现自动构建、部署，形成「生成→测试→修复→部署」的完整DevOps闭环

三、当前核心亮点（总结，用于面试/答辩）

- 全真实闭环：无模拟数据，所有Agent真实调用大模型、真实执行操作，可直接落地
- 规范统一：所有Agent遵循「枚举→Agent→提示词文件」的标准结构，便于维护和扩展
- 轻量无依赖：RAG采用纯Java实现，无需额外安装数据库插件（如pgvector），可直接运行

- 可扩展性强：新增Agent只需新增枚举、Agent类、提示词文件，无需修改核心调度逻辑

(注：文档部分内容可能由 AI 生成)